PROJECT BRIEF · DISTRIBUTED SYSTEMS · FINANCIAL ENGINEERING · SRE

Distributed Pricing Service

A front-office pricing and CVA platform in miniature: a C++20 Monte Carlo library behind a Python service, distributed across Kubernetes worker pools on Redis Streams and Cassandra, managed in Rancher, observed end to end, and run through chaos tests that kill pods, databases and a whole node under load.

Sukhmanpreet Singh Aulakh · Oakville, ON Case study: sukhmanaulakh18.github.io/distributed-pricing-service

Code: github.com/SukhmanAulakh18/distributed-pricing-service

99.992%
of 76,481 prices served through 11 faults, including a whole node stopped

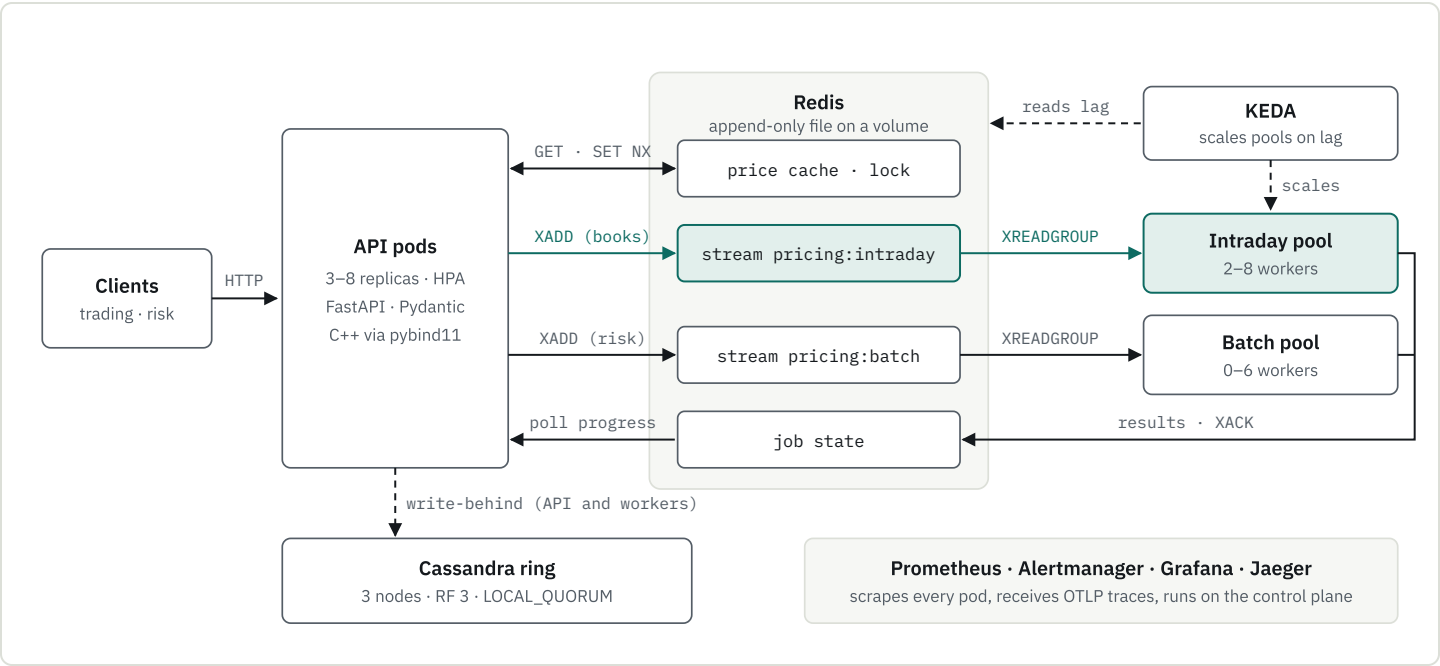
29 / 30 books
completed, with 0 of 58,000 trades failed; exposure runs bit-identical

5.17 → 0.32 s
intraday book during a batch run, after splitting priority streams

2,295 → 7,335
trades/s on 1 → 8 workers; 50,000-trade book in 6.9 s

1,009× variance cut
Asian control variate: 198 paths match 200,000

176 vs 46
pricings/s from 8 Python threads, GIL released vs held (pybind11)



Financial engineering

- Black-Scholes, European and arithmetic Asian Monte Carlo with antithetic variates, a geometric control variate and pathwise Greeks
- Exposure engine: correlated GBM, EE, PFE (exact top-k merge), EPE and unilateral CVA
- Every model tested against an independent reference (textbook values, parity, closed forms)

Reliability and SRE

- SLOs with multi-window burn-rate alerts; 14 alerts, each with an owner and a runbook link
- Chaos runs that found three real problems; fixes measured; one still open and documented
- OpenTelemetry traces across processes; runbook with 14 incidents; triage CLI

Distributed systems

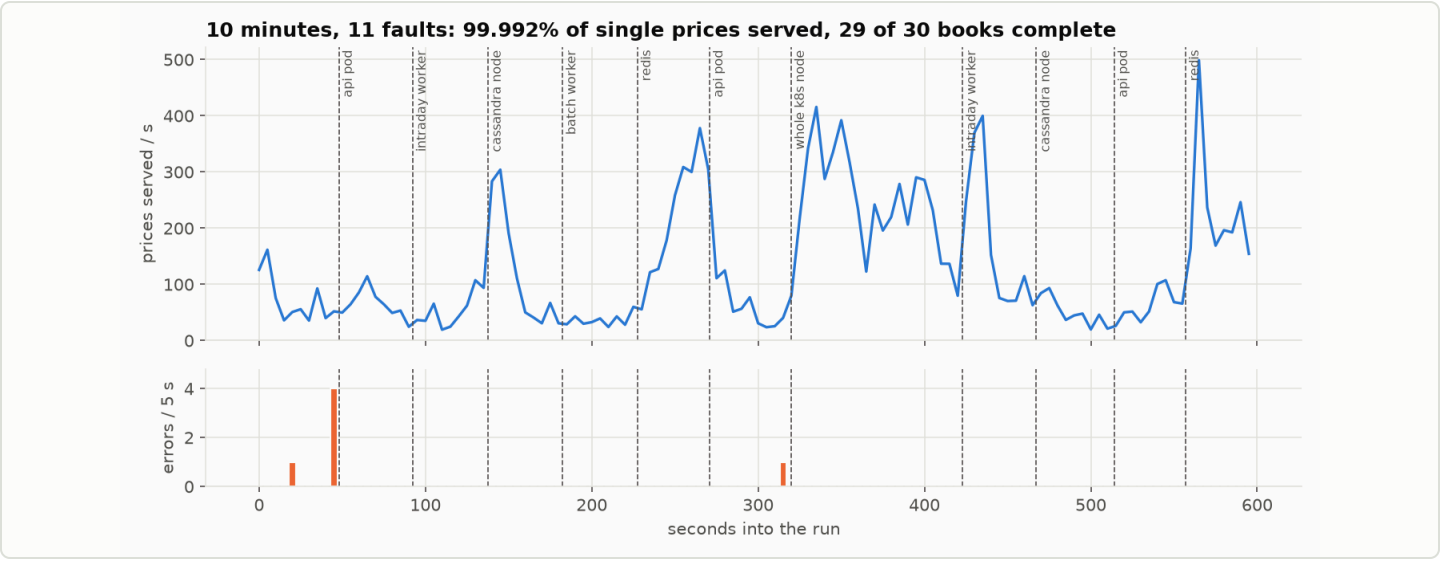
- Redis Streams with consumer groups, reclaim of dead workers' tasks, poison handling, separate intraday and batch priority
- Counter-based RNG keyed by request hash: exact sharding, safe caching, idempotent re-runs
- Cassandra modelled query-first (RF 3, LOCAL_QUORUM, token-aware), written behind

Platform and DevOps

- 4-node kind cluster over 3 zones, imported into Rancher; PDBs, zone spread, 30 s eviction
- KEDA scaling on queue lag (batch from zero), HPA for the API; Kustomize base and overlays
- CI: C++ (GCC, Clang), Python, ASan/UBSan, TSan, Valgrind, kubeconform, promtool, amtool

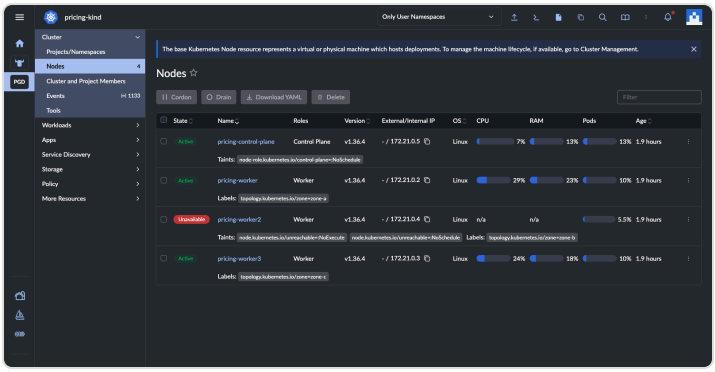
C++20 · pybind11 · Python · FastAPI · Pydantic · Redis · Cassandra · Kubernetes · KEDA · Rancher · Docker · Prometheus · Grafana · Jaeger

How it behaves under failure, and how you would know

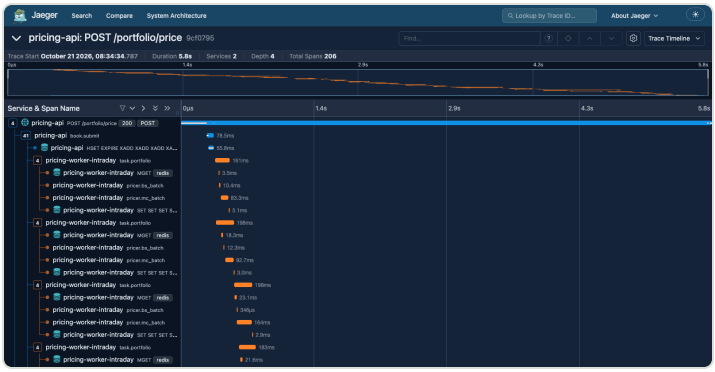


Ten minutes, 11 faults. Prices served per second (top) and errors per 5 s (bottom); dashed lines are faults. An interactive replay is on the case-study site.

OPERATION OR FAILURE	WHAT THE SYSTEM DOES	EVIDENCE
Single price	Cache read; on a miss, one pod computes under a lock, with the GIL released	~1 ms hit; 4 ms p50 miss (200k paths)
Book (intraday)	Split into tasks on the intraday stream; one cache read, one C++ batch and one write per task	One trace across 3 pods (206 spans)
Exposure run (batch)	Scenario blocks on the batch stream; KEDA scales from 0; exact merge	604 M revaluations in 2.7 s; identical on 1-8 workers
Worker killed	Unacked task reclaimed after 10 s idle and re-priced, same answer	0 of 10,000 trades lost
Redis killed	Circuit opens; pricing continues uncached; queues survive on the AOF	0 errors; RedisCircuitOpen → platform
Cassandra node killed	Quorum writes continue on 2 of 3 replicas	0 failed writes; ReplicaDown → database
Kubernetes node stopped	NotReady at ~50 s, pods evicted at ~80 s and rescheduled across zones	NodeNotReady → platform; 1 book lost (open item)
Load spike	Admission control returns 503 + Retry-After; HPA and KEDA add pods	API 3 → 8 pods in 26 s, 0 errors



Rancher during the node outage: the node is Unavailable and tainted.



One book as one trace across the API and workers.

Found and fixed by the chaos runs: books lost while a dead node was still in rotation (fixed with an idempotent retry); monitoring that died with the node it watched (moved to the control plane); an SLO panel that read "No data" when there were zero errors. **Still open:** one book in 30 lost during a two-minute node outage; the fix is an idempotency key the client polls on.